MAD-2 Project Report

Author

Name: Anirudh Venkateswaran

Roll no. : 23f3003601

Email : 23f3003601@ds.study.iitm.ac.in

I am currently pursuing a **B.Tech in Computer Science and Engineering at NIT Rourkela** and simultaneously enrolled in the **BS in Data Science and Applications program at IIT Madras**. This project was developed as part of the **Modern Application Development-2 (MAD-2)** course requirements and demonstrates the development of a modern web application using a REST-based backend and reactive frontend architecture.

AI / LLM Usage

During the development of this project, AI-assisted tools such as ChatGPT and Claude were used primarily as **development assistants for debugging, code suggestions, and architectural clarification**. The use of these tools was limited to assisting with boilerplate generation, resolving framework compatibility issues, and improving development productivity. All major decisions related to system architecture, database design, API structure, and feature implementation were performed independently by me.

The AI tools functioned mainly as a **pair-programming assistant**, helping accelerate development while ensuring that I retained full control over system design and implementation logic. The majority of the conceptual understanding, system organization, and debugging processes were handled manually during the development cycle.

Based on the project component analysis, the approximate AI-assisted contribution for the MAD-2 implementation remained **within the allowed limit**. The backend components involving Flask APIs and database interaction account for approximately **20%**, while the frontend components implemented using Vue.js and JavaScript contribute approximately **30%**. The styling and UI implementation using Bootstrap represent around **8%**, while authentication mechanisms and CRUD logic contribute about **15%**. Additional functionality such as charts, external API interactions, and auxiliary features contribute roughly **7%**. Overall, the total AI-assisted contribution in the project remains **within the acceptable maximum usage threshold defined for the course**.

Description

The objective of this project is to design and implement a **role-based Hospital Management System web application** that enables different categories of users to interact with a centralized healthcare management platform. The application supports three primary roles: **Admin, Doctor, and Patient**. Each role is provided with specific functionalities designed to streamline hospital operations and improve interaction between medical staff and patients.

The Admin is responsible for managing doctors, patients, departments, and appointments. Doctors can manage their schedules, record treatment details, and review patient history. Patients can search for doctors based on specialization, book appointments, and view treatment records. The system is implemented using a **Flask REST API backend and a Vue.js-based frontend**, creating a responsive and interactive web application architecture.

Technologies Used

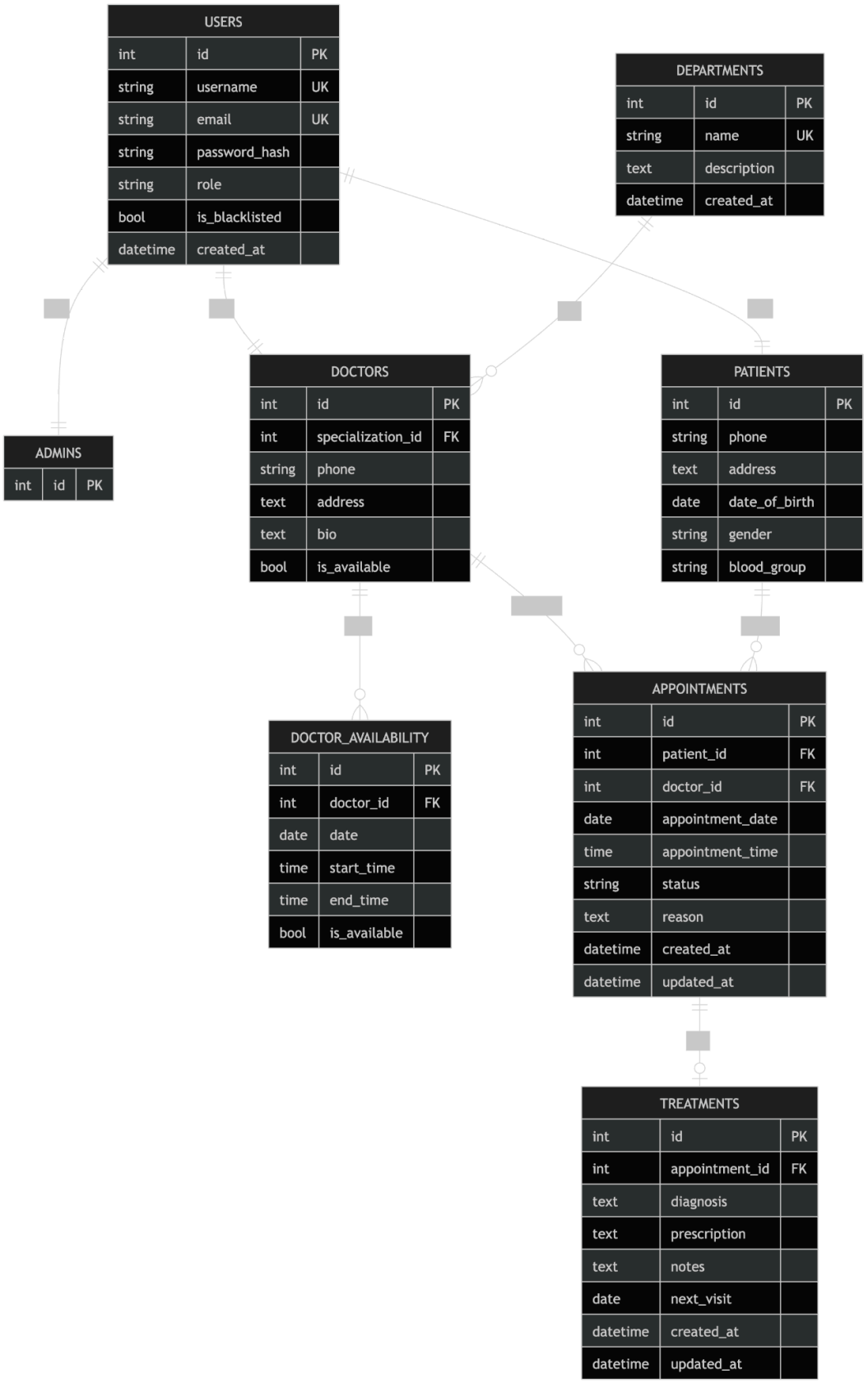
The backend of the application is implemented using **Flask**, which serves as the primary framework for building RESTful APIs and handling server-side logic. Database interactions are managed using **SQLAlchemy ORM**, which simplifies relational database modeling and query handling. The application uses **SQLite** as the underlying database system, providing a lightweight and easy-to-deploy storage solution.

Authentication and session management are handled through **Flask-Login**, while **Werkzeug** is used for secure password hashing and verification. **Flask-CORS** enables cross-origin resource sharing to allow communication between the frontend and backend during development. Email functionality and notifications are implemented using **Flask-Mail**.

The frontend interface is developed using **Vue.js 3**, which provides a reactive component-based architecture for building dynamic user interfaces. **Axios** is used to handle API communication between the frontend and backend services. The visual design and layout of the application are implemented using **Bootstrap**, which provides responsive styling components. Data visualization features on the admin dashboard are implemented using **Chart.js**, enabling graphical representation of appointment statistics and departmental distributions.

To enhance system performance and scalability, **Redis** is used as a caching layer for frequently accessed data such as department lists and dashboard statistics. **Celery** is integrated to support asynchronous task execution, including scheduled reminder notifications and periodic report generation.

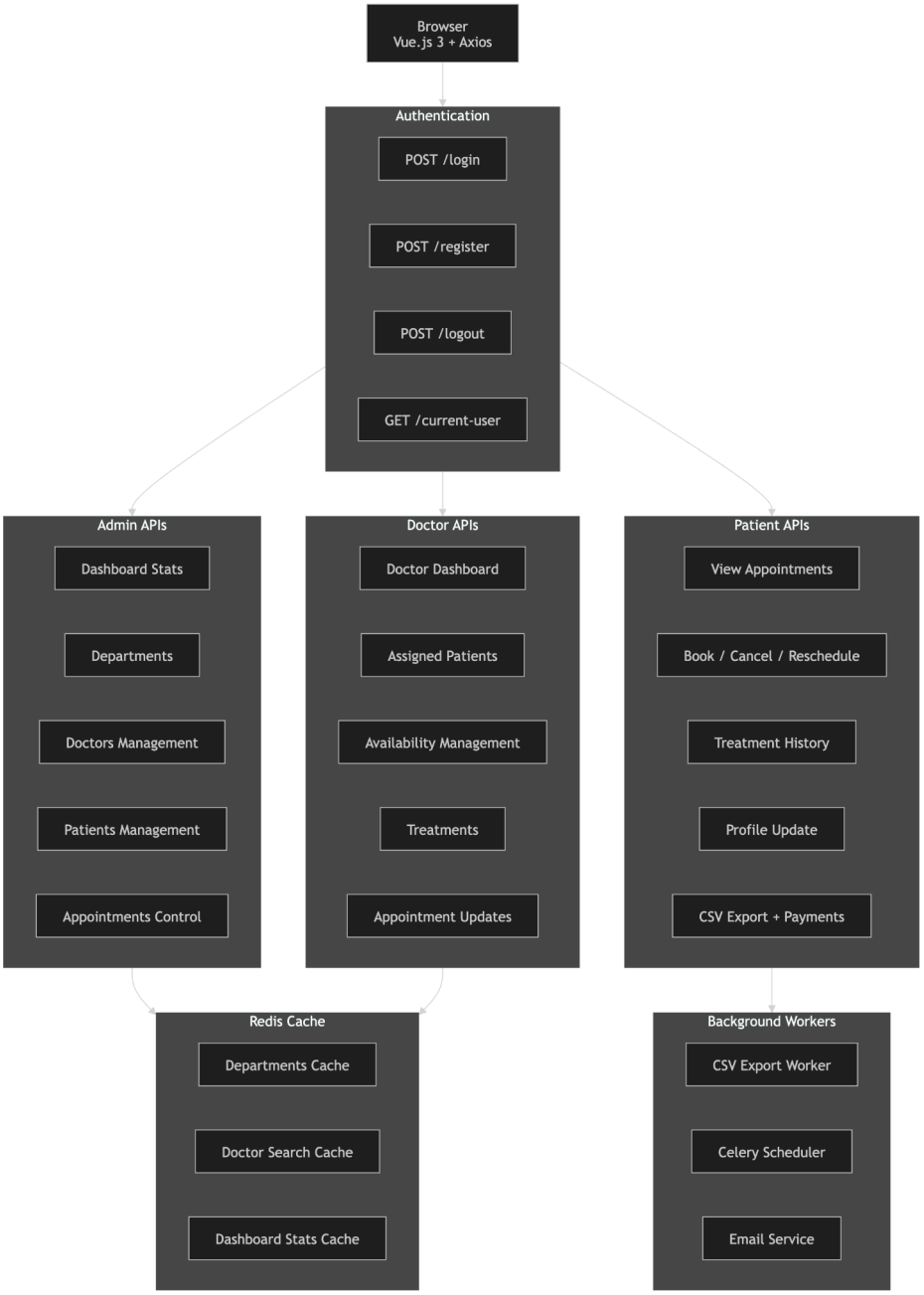
DB Schema Design



The database schema follows a relational structure implemented through SQLAlchemy models. A central **Users table** stores authentication information and user roles. Separate tables for **Doctors** and **Patients** store role-specific information and reference the user table through foreign key relationships. A **Departments table** organizes doctors by specialization.

The **Appointments table** acts as the main relationship between doctors and patients, storing appointment date, time, and status. A **Treatments table** extends appointment records by storing diagnosis details, prescriptions, and medical notes for completed consultations. This design ensures clear separation of responsibilities, efficient relational mapping, and normalized storage of healthcare data.

API Design and Architecture



The application exposes a set of **RESTful APIs under the `/api` namespace** that allow the frontend to interact with the backend. These APIs handle authentication, doctor and patient management, appointment booking, and treatment record management. Role-based access control ensures that each user can only access authorized operations. The APIs exchange data using JSON format and are implemented using Flask route handlers with validation and error handling. The complete API specification is documented separately using a **YAML API definition file** submitted with the project.

The system follows a **client-server architecture**, where the Vue.js frontend communicates with the Flask backend through REST APIs. The backend manages authentication, business logic, and database interactions, while the frontend is responsible for rendering user interfaces and managing application state. The project structure separates configuration files, models, API routes, and frontend templates to maintain modularity.

Core features of the system include role-based authentication, appointment booking, doctor availability management, treatment record storage, and dashboard analytics. Admin users can manage system entities and monitor activity through charts, doctors can track appointments and patient history, and patients can schedule consultations and view treatment information. Additional features such as caching, background tasks, and data visualization improve system performance and usability.

Video

The project demonstration video explaining the system architecture, major features, and workflow is available at the following link:

Video Link:
https://drive.google.com/file/d/1UTzAqOJD_p_G2w3ty9tmY8oM0_hqGII3/view?usp=sharing

The video briefly demonstrates the application interface, role-based functionalities, appointment management workflow, and the overall architecture of the system.